

Lumen — Portfolio Documentation

AI-Based Web Accessibility Checker

Repository: <https://github.com/balisikh/ai-based-web-accessibility-checker>

Local app: <http://localhost:4376>

Document date: **2026-08-04**

Assistive findings only — this tool is **not** a legal accessibility certificate or formal conformance audit.

Overview

Lumen scans a public URL for WCAG-oriented accessibility issues, returns a clear score, severity breakdown, and links to rule help. Optional AI tips suggest fixes for top issues. A separate **batch dashboard** shows Pass/Fail results for 28 well-known public websites.

What it does (end-to-end)

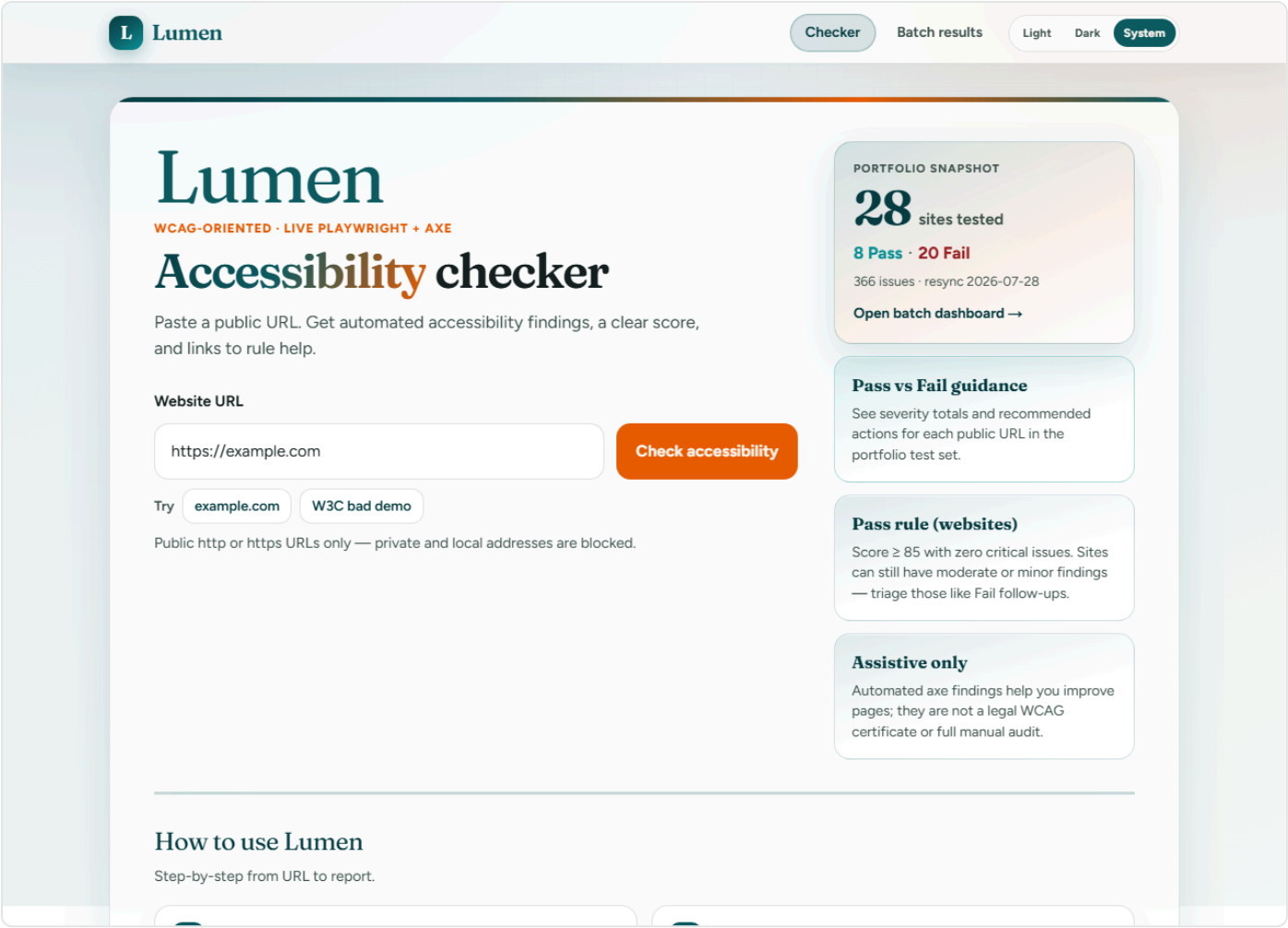
1. Paste a public website URL on the home page
2. Lumen opens the page in a headless browser (Chrome via Playwright)
3. **axe-core** checks WCAG A/AA-oriented rules
4. You get a **score**, severity breakdown, and issue details
5. Optionally, **AI tips** explain top issues and suggest fixes
6. Export a **JSON** report

Technology stack

Layer	Technology
Frontend / API	Next.js 16 (React) + TypeScript
Scanner	Playwright (Chrome) + axe-core
Data	Postgres or local PGLite
AI (optional)	OpenAI-compatible Chat Completions API
CI/CD	GitHub Actions, Docker, GHCR

UI screenshots (styled layout)

Home — accessibility checker (hero)



How to use Lumen

Step-by-step from URL to report.

- 1

Enter a public URL
Paste an http or https link in the field above (for example https://example.com), or pick a Try example.
- 2

Check accessibility
Click Check accessibility to scan that single page. Localhost and private network addresses are blocked.
- 3

Wait for progress
Follow the status steps while Lumen fetches the page, runs rules, optional AI tips (when enabled), and calculates a score. Most public pages finish within about a minute.
- 4

Review findings
Check the score and severity counts. Select an issue to see WCAG mapping, HTML snippet, rule help, and AI guidance when enabled on this server. Filter by severity if the list is long.
- 5

Export JSON (optional)
On the results screen, use Export JSON to download the report for tickets, docs, or automation.

How Lumen checks a page

AI tips off

- 1

Public websites
Use http or https links that anyone can open on the internet.
✓ https://example.com ✗ localhost or 127.0.0.1 ✗ 192.168.x.x
- 2

Real browser scan
Automated Chrome loads the page in the background, like a real visitor.
- 3

Accessibility rules
Industry-standard checks (axe-core) for common WCAG Level A and AA requirements.
- 4

Optional AI tips
Not enabled on this Lumen server — you still get rule findings, WCAG mapping, and a score.

Website batch results

- + What URLs can I use?
- + What are the limits of a scan?
- + How is the score calculated? What does Pass / Fail mean?
- + What are AI tips? Why are they off?
- + Why did my scan fail or say rate limited?
- + What happens to my URL and scan data?

Batch results — 28 sites (overview)

L Lumen

CheckerBatch resultsLightDarkSystem

PORTFOLIO TEST SET

Run a live scan

Website batch results

Static snapshot (last full live resync **2026-07-28**) — loads instantly and does not run scans. Live checks use the accessibility checker.

TESTED
28

PASSED
8
4 clean · 4 with issues

FAILED
20

TOTAL ISSUES
366

CRITICAL
126

SERIOUS
30

MODERATE
189

MINOR
21

Pass: score ≥ 85 and critical = 0. Fail: otherwise. Pass and Fail rows both use **Notes & recommendations** when there are issues to triage. Source: TEST_RESULTS.md in the repo.

#	WEBSITE	SCORE	CRITICAL	SERIOUS	MODERATE	MINOR	TOTAL	RESULT	NOTES & RECOMMENDATIONS
1	Google UK BENCHMARK PASS CLEAN SCAN	100	0	0	0	0	0	PASS	MAINTAIN PASS NOTE Perfect automated score (0 issues). Use as a clean baseline when comparing other scans. Re-scan after major Google UI changes — automated results can shift. Keep manual spot-checks; Pass is not a formal WCAG certificate.
2	YouTube CRITICAL BLOCKER	0	4	0	0	0	4	FAIL	4 critical issues 4 critical issues; score 0 (need ≥ 85) 4 issues — work through recommendations below RECOMMENDED ACTIONS Fix 'aria-allowed-attr' on main nav links (remove or correct invalid ARIA). Re-scan in Lumen and use Rule help on each critical issue. Target zero critical issues, then re-check score ≥ 85. Re-scan in Lumen for live rule detail.
3	BBC Weather	54	1	0	0	0	1	FAIL	

3	BBC Weather Southall	54	1	0	3	0	4	FAIL	1 critical issue 1 critical issue; score 54 (need ≥ 85) 4 issues — work through recommendations below	RECOMMENDED ACTIONS Fix critical 'aria-required-attr' first. Resolve 'aria-hidden-focus' and 'color-contrast' (serious) on weather UI. Re-scan after BBC deploys changes (page content changes often). Re-scan in Lumen for live rule detail.
4	BBC iPlayer	86	0	0	2	0	2	PASS	2 issues — batch Pass; triage below (keep critical at 0)	RECOMMENDED ACTIONS NOTE Pass with 2 moderate issues (score 86). Open each moderate issue in Lumen Rule help and fix when convenient. Re-scan to confirm critical stays at 0 and score stays ≥ 85. Prioritize any new critical findings before release. Still a batch Pass — re-scan in Lumen after fixes so critical stays 0
5	BBC News	93	0	0	1	0	1	PASS	1 issue — batch Pass; triage below (keep critical at 0)	RECOMMENDED ACTIONS NOTE Pass with 1 moderate issue (score 93). Triage the moderate finding; low effort may lift score further. Re-scan after BBC template or component updates. Export JSON if tracking regressions in a ticket. Still a batch Pass — re-scan in Lumen after fixes so critical stays 0
6	Disney+ UK	100	0	0	0	0	0	PASS	CLEAN SCAN	MAINTAIN PASS NOTE Clean scan — 100 score, zero issues. Re-scan periodically; streaming sites update often. Use as a Pass reference alongside Google UK / Maps. Still validate key flows manually (keyboard, captions etc.).
7	GitHub balisikh	100	0	0	0	0	0	PASS	CLEAN SCAN	MAINTAIN PASS NOTE Profile page Re-scan if GitHub changes profile markup or ARIA patterns. Scan other URLs separately in testing org/repo pages. Maintain zero critical on re-scan to keep batch Pass.
8	ChatGPT	50	1	1	1	1	4	FAIL	1 critical issue 1 critical issue; score 50 (need ≥ 85) 4 issues — work through recommendations below	RECOMMENDED ACTIONS Correct unsupported ARIA ('aria-allowed-attr', critical). Improve text contrast ('color-contrast', serious). Re-scan; export JSON to track before/after. Re-scan in Lumen for live rule detail.
9	Spotify Web Player	0	82	0	2	16	100	FAIL	82 critical issues 82 critical issues; score 0 (need ≥ 85) 100 issues — work through recommendations below	RECOMMENDED ACTIONS NOTE Re-scan: app shell / login — confirm in UI Confirm URL and cookies (open.spotify.com vs marketing landing). Triage 82 critical findings — fix invalid ARIA/roles first. Re-scan after stable page load; compare to prior Pass (100/0).

Re-scan in Lumen for live rule detail.

10 Google Maps 100 0 0 0 0 0 PASS

CLEAN
SCAN

MAINTAIN PASS

NOTE Clean scan — 100 score, zero issues.

Maps is heavy JS — re-scan after Google updates the app shell.

Compare against other clean Pass sites if scores diverge.

Results — sample layout

L Lumen

Checker

Batch results

Light

Dark

System

[← New scan](#)

Lumen

Results

<https://www.w3.org/WAI/demos/bad/before/home.html>

42

SCORE

Needs work

BATCH: FAIL

Critical: 3. 3 critical issues; score 42 (need ≥ 85)

3 critical issues — address these before lower-severity items, even if the score looks acceptable.

Score **0–100** from automated issue counts. **Needs work** means 42 (85+ is Strong). In our website test batches, **Pass** also requires zero critical issues.

CRITICAL

3

SERIOUS

2

MODERATE

5

MINOR

1

CRITICAL Blocks many users — fix first.

SERIOUS Major barriers for some users.

Filter severity

All ▼

Export JSON

New scan

Assistive findings only — not a legal accessibility certificate.

CRITICAL

Images must have alternate text: decorative banner missing alt

SERIOUS

CRITICAL

Images must have alternate text

WCAG 1.1.1

Rule image-alt

L

Lumen

Checker

Batch results

Light

Dark

System

Lumen

WCAG-ORIENTED · LIVE PLAYWRIGHT + AXE

Accessibility checker

Paste a public URL. Get automated accessibility findings, a clear score, and links to rule help.

Website URL

https://example.com

Check accessibility

Try

example.com

W3C bad demo

Public http or https URLs only — private and local addresses are blocked.

Batch snapshot (2026-08-04)

Static dashboard at `/batch` — loads instantly from committed or live JSON data.

Metric	Value
Websites tested	28
Passed	8 (4 clean · 4 with issues)
Failed	20
Total issues	330
Last live refresh	28/28 OK (~88 seconds, parallel background job)

Pass / Fail rule

Result	Rule
Pass	Score ≥ 85 and Critical issues = 0
Fail	Score < 85 or Critical issues ≥ 1

Severity totals (all 28 scans)

Critical	Serious	Moderate	Minor
46	37	217	30

Feature reference

Each feature below follows the same structure: **what it is**, **why we use it**, **key purpose**, **how we implemented it**, and **how to use it**.

1. Live URL scanning (Playwright + axe-core)

What it is	A real browser scan of one public URL. Playwright launches headless Chrome, loads the page, and axe-core runs WCAG 2.x A/AA tagged rules.
Why we use it	Static HTML checks miss client-rendered content. axe-core is industry-standard for automated accessibility testing and maps to WCAG success criteria.
Key purpose	Give developers a fast, trustworthy signal on real page behaviour — not just source code.
How we implemented it	<code>web/src/lib/live-scan.ts</code> — shared browser instance, navigation fallbacks (<code>DOMContentLoaded</code> → <code>load</code> → <code>commit</code>), axe tags <code>wcag2a</code> , <code>wcag2aa</code> , <code>wcag21a</code> , <code>wcag21aa</code> , <code>wcag22aa</code> . Violations mapped in <code>axe-mapper.ts</code> . Pipeline wired from <code>POST /api/scans</code> .
How to use it	Home page → paste URL → Check accessibility . Progress shows fetching → rendering → rule analysis. Results appear when complete.

2. Scan workflow UI (home → progress → results)

What it is	The main user journey: URL input, live progress, score panel, issue list, and detail view.
Why we use it	Scanning can take 10–60 seconds. Clear states reduce anxiety and match the “fast first report” product goal.
Key purpose	Make accessibility checking feel simple for non-experts while still showing enough detail for developers.
How we implemented it	<code>ScanExperience.tsx</code> (client) polls <code>GET /api/scans/:id</code> . Server components pass batch snapshot context. Copy from <code>how-to-use.ts</code> and <code>product-copy.ts</code> . Fixture route <code>/fixtures/results</code> for layout QA without live scans.
How to use it	Open <code>/</code> . Enter a URL or click a Try example chip. Watch progress. Click an issue for rule help and (if enabled) AI tips.

3. Scoring and Pass / Fail rules

What it is	A 0–100 score derived from issue severities, plus a binary Pass/Fail gate for the batch dashboard.
Why we use it	Teams need one number for triage and a clear portfolio benchmark across 28 sites.
Key purpose	Deterministic scoring (rules find issues; score is formula-based, not AI-generated).
How we implemented it	<code>store.ts</code> — <code>scoreFromIssues()</code> , <code>countBySeverity()</code> . Pass rule in <code>website-pass-fail.ts</code> : Pass if score ≥ 85 and critical = 0. Documented in <code>product-copy.ts</code> and home FAQs.
How to use it	Results panel shows score + severity chips. Batch table shows Pass/Fail per site. Hover or read Common questions on home for the full rule.

4. URL validation and SSRF protection

What it is	Server-side checks that block scanning of private networks, localhost, and non-public hosts.
Why we use it	Without this, a scanner could be abused to probe internal IPs (SSRF).
Key purpose	Security by default — only public web pages are allowed.
How we implemented it	<code>ssrf.ts</code> — hostname blocklist, DNS resolve, reject private/reserved IP ranges. Called from <code>live-scan.ts</code> before navigation. User-facing errors in FAQs (<code>home-faqs.ts</code>).
How to use it	Use normal <code>https://</code> public URLs. Localhost and <code>192.168.x.x</code> are rejected with a clear message.

5. Rate limiting (live scans and batch refresh)

What it is	Per-IP limits on how often clients can start expensive operations.
Why we use it	Playwright scans are CPU/RAM heavy. Limits prevent abuse and keep the app responsive for everyone.
Key purpose	Protect the server without requiring user accounts.
How we implemented it	<code>rate-limit.ts</code> — in-memory sliding window. <code>POST /api/scans</code> : default 5/min (production). Batch refresh: <code>batch-refresh-config.ts</code> — 5 per 10 min locally, 1/hour in production CI hosts. Returns 429 + <code>Retry-After</code> .
How to use it	If you hit the limit, wait for the retry time shown. Override via <code>RATE_LIMIT_MAX</code> , <code>BATCH_REFRESH_RATE_LIMIT_*</code> in <code>.env.local</code> for self-hosted setups.

6. Persistence (PGlite and Postgres)

What it is	Scan records and issues stored in a database so results survive page reloads and server restarts.
Key purpose	Shareable scan URLs, JSON export, and audit trail without login.
Why we use it	In-memory-only storage would lose data on every deploy; Postgres is needed for production.
How we implemented it	<code>db/</code> schema + async scan store. Default: PGlite at <code>web/data/lumen-pg</code> . Production: set <code>DATABASE_URL</code> for Postgres. Health check at <code>GET /api/health</code> .
How to use it	No setup locally — PGLite creates files automatically. For production, set <code>DATABASE_URL</code> in host env (see <code>web/DEPLOY.md</code>).

7. JSON export

What it is	Download of the full scan report as structured JSON.
Why we use it	Developers and QA tools need machine-readable output for tickets, CI, or archival.
Key purpose	Primary MVP export format (PDF export is roadmap).
How we implemented it	<code>GET /api/scans/:id/export?format=json</code> — issues, score, metadata. UI button on results panel.
How to use it	After a scan completes → Export JSON on the results view.

8. Optional AI tips

What it is	LLM-generated explanation and suggested fix for the top N issues by severity.
Why we use it	axe tells you *what* failed; AI helps explain *why it matters* and *how to fix it* for developers who are new to a11y.
Key purpose	Hybrid detection: rules find issues; AI enriches — scoring stays deterministic.
How we implemented it	<code>ai-enrichment.ts</code> — OpenAI-compatible API. Env: <code>AI_API_KEY</code> / <code>OPENAI_API_KEY</code> , <code>AI_MODEL</code> , <code>AI_MAX_ISSUES</code> . Failures never block the report. <code>GET /api/config</code> exposes <code>aiTipsEnabled</code> without secrets.
How to use it	Add key to <code>web/.env.local</code> , restart server. Top issues show AI tip sections when enabled.

9. Batch results dashboard (28 sites)

What it is	A pre-built Pass/Fail table for 28 public websites (Google, BBC, Netflix, W3Schools, etc.).
Why we use it	Demonstrates Lumen on real-world pages instantly — no waiting for 28 live scans on every visit.
Key purpose	Portfolio proof and regression benchmark across diverse site types (SPAs, media, e-commerce, docs).
How we implemented it	<code>website-batch-results.ts</code> — committed snapshot. <code>/batch</code> page renders summary tiles + sortable table. Fail guidance in <code>website-batch-fail-guidance.ts</code> . Unified date via <code>getBatchSnapshot()</code> in <code>batch-snapshot-store.ts</code> .
How to use it	Nav → Batch results . Review Pass/Fail, scores, severities. Click Run a live scan to test your own URL.

10. Refresh snapshot (background batch rescan)

What it is	A button on <code>/batch</code> that live-rescans all 28 sites and saves a runtime JSON snapshot.
Why we use it	Keeps the dashboard up to date without a manual CLI run or waiting for weekly CI.
Key purpose	Fast refresh (~under 2 minutes) without blocking the rest of the app.
How we implemented it	<code>POST /api/batch/refresh</code> → 202 Accepted , background job in <code>batch-refresh-job.ts</code> . Parallel rescans (<code>BATCH_RESCAN_CONCURRENCY=2</code>) in <code>batch-live-rescan.ts</code> . Per-site 90s timeout. Progress polling via <code>GET /api/batch/refresh</code> . UI: <code>BatchRefreshButton.tsx</code> shows <code>Scanning SiteName (N/28)...</code> . Saves <code>web/data/batch-live-snapshot.json</code> . Enabled in dev or with <code>BATCH_REFRESH_ENABLED=1</code> .
How to use it	<code>/batch</code> → Refresh snapshot . Watch progress. Page reloads when done. Tune speed via <code>.env.local</code> (see <code>web/README.md</code>).

11. Batch sync CLI and weekly CI rescan

What it is	Automated pipeline to rescan all 28 sites and commit updated scores to git.
Why we use it	Deployed sites read committed TypeScript data — runtime JSON alone does not survive fresh deploys.
Key purpose	Keep production and git in sync with real-world site changes over time.
How we implemented it	<code>npm run batch:sync</code> = <code>batch:rescan</code> → <code>batch:apply-rescan</code> → <code>validate:batch</code> . GitHub Actions: <code>.github/workflows/batch-rescan.yml</code> — Sunday 04:00 UTC + manual dispatch; commits <code>website-batch-results.ts</code> when changed.
How to use it	Local: <code>cd web && npm run batch:sync</code> , then commit. CI: Actions → Batch rescan → Run workflow.

12. Theme system (Light / Dark / System)

What it is	User-selectable colour theme with system preference detection.
Why we use it	Accessibility tools should respect user contrast preferences; dark mode reduces eye strain for long review sessions.
Key purpose	WCAG-aligned product UI — the checker itself should be usable.
How we implemented it	CSS variables in <code>globals.css</code> — <code>[data-theme="light"]</code> , <code>dark</code> , <code>system</code> via <code>prefers-color-scheme</code> . Theme toggle in header. Batch page uses readable table/card styles in both themes.
How to use it	Header → Light , Dark , or System . Preference persists in <code>localStorage</code> .

13. Responsive layout and viewport CI

What it is	Layout adapts from mobile (320px) to desktop (1440px+) with device-tier gutters and breakpoints.
Why we use it	Teams review scans on phones and tablets; overflow bugs break real usage.
Key purpose	One codebase, five layout tiers — mobile cards, tablet batch cards, laptop/desktop tables.
How we implemented it	Breakpoint tokens in <code>globals.css</code> (<code>--bp-mobile 480</code> , <code>--bp-tablet 768</code> , <code>--bp-laptop 1024</code> , <code>--bp-desktop 1440</code>). <code>npm run test:responsive</code> — Playwright checks overflow at 320, 390, 768, 1024, 1280, 1440 on <code>/</code> , <code>/batch</code> , <code>/fixtures/results</code> . CI job responsive-viewport .
How to use it	Resize browser or use DevTools device mode. CI runs automatically on push to <code>main</code> .

14. SEO (search visibility)

What it is	Metadata, sitemap, robots.txt, Open Graph, and JSON-LD for public deployment.
Why we use it	Portfolio and product discoverability when deployed to a public URL.
Key purpose	Correct canonical URLs, social previews, and crawler guidance.
How we implemented it	<code>seo.ts</code> , <code>layout.tsx</code> metadata, <code>/sitemap.xml</code> , <code>/robots.txt</code> , <code>/icon.svg</code> . Requires <code>NEXT_PUBLIC_SITE_URL</code> in production (<code>web/DEPLOY.md</code>).
How to use it	Set <code>NEXT_PUBLIC_SITE_URL</code> on deploy. Submit sitemap in Google Search Console.

15. Continuous integration (GitHub Actions)

What it is	Automated checks on every push/PR to <code>main</code> .
Why we use it	Catch lint, type, build, and snapshot regressions before merge.
Key purpose	Confidence that production build and batch data stay valid.
How we implemented it	<code>.github/workflows/ci.yml</code> — ESLint, <code>typecheck:scripts</code> , <code>validate:copy</code> , <code>validate:batch</code> , <code>next build</code> , responsive viewport job. Badge in root README.
How to use it	<code>cd web && npm run ci</code> locally before push. Fix any red checks in Actions tab.

16. Continuous deployment (Docker + GHCR)

What it is	Docker image build and push after CI passes on <code>main</code> .
Why we use it	Playwright needs a worker-capable host with Chrome — container standardizes deploy.
Key purpose	Repeatable production deploy path.
How we implemented it	<code>web/Dockerfile</code> , <code>.github/workflows/cd.yml</code> → GHCR <code>ghcr.io/<owner>/lumen-web</code> . Optional <code>RENDER_DEPLOY_HOOK</code> secret. Documented in <code>web/DEPLOY.md</code> .
How to use it	Configure host secrets (<code>DATABASE_URL</code> , <code>NEXT_PUBLIC_SITE_URL</code> , optional AI key). Pull image or connect deploy hook.

17. Centralized product copy

What it is	Single source of truth for UI strings, FAQs, Pass/Fail rule, and disclaimers.
Why we use it	Prevents README and UI drift; FAQs stay consistent across home and docs.
Key purpose	Maintainability — change copy once, validate everywhere.
How we implemented it	<code>product-copy.ts</code> , <code>how-to-use.ts</code> , <code>home-faqs.ts</code> , <code>try-examples.ts</code> . <code>npm run validate:copy</code> fails CI if README duplicates in-app steps/FAQs.
How to use it	Edit copy in <code>web/src/lib/</code> . Run <code>npm run validate:copy</code> . Do not duplicate long FAQ text in README.

18. Demo mode (no browser)

What it is	Pre-canned scan findings without Playwright or network access.
Why we use it	CI, low-RAM machines, or demos when Chrome is unavailable.
Key purpose	UI development and testing without live scans.
How we implemented it	<code>USE_DEMO_SCAN=1</code> env flag — sample issues returned instead of <code>live-scan.ts</code> .
How to use it	<code>\$env:USE_DEMO_SCAN="1"; npm run dev</code> (PowerShell) or <code>USE_DEMO_SCAN=1 npm run dev</code> .

19. Local development tooling

What it is	Scripts and defaults for stable development on Windows and low-RAM machines.
Why we use it	Next.js 16 Turbopack could crash compiling large <code>globals.css</code> on low memory, leaving unstyled HTML.
Key purpose	Reliable local demos for portfolio and interviews.
How we implemented it	Webpack forced via <code>--webpack</code> on <code>dev</code> / <code>build</code> . <code>start-local.ps1</code> , <code>fix-styles.ps1</code> , <code>build:low-mem</code> / <code>start:low-mem</code> (2 GB heap). Troubleshooting in <code>web/README.md</code> .
How to use it	If page shows plain white text: <code>cd web; .\scripts\fix-styles.ps1</code> . For production-like local: <code>npm run build:low-mem && npm run start:low-mem</code> .

MVP feature checklist

Area	Status
Scan UI (home → progress → results)	Done
Live Playwright + axe-core analysis	Done
URL validation + SSRF protections	Done
Per-IP rate limiting	Done
Persistence (Postgres or PGlite)	Done
JSON export	Done
Optional AI enrichment	Done
Anonymous use (no login)	Done
Batch dashboard (28 sites)	Done
Refresh snapshot (background)	Done
Weekly CI batch rescan	Done
Responsive layout + viewport CI	Done
SEO (sitemap, OG, JSON-LD)	Done
Docker + CD workflow	Done
PDF export / accounts / multi-page crawl	Not yet

Quick start

```
cd web
npm install
npm run dev
```

Open **http://localhost:4376**

On memory-limited Windows:

```
npm run build:low-mem
npm run start:low-mem
```

Enable batch refresh locally (`.env.local`):

Local development note — plain-text UI incident (Aug 2026)

During local testing, the app briefly appeared as **unstyled plain text** (nav labels run together). The design was **not removed** — CSS failed to load from a stale `.next` build.

Fix: webpack by default, `fix-styles.ps1`, confirm CSS **200** in DevTools → Network. After clean rebuild, UI matched screenshots again.

Disclaimer

Lumen helps teams find and understand accessibility issues faster. It does **not** replace a professional audit, and results should not be treated as legal proof of WCAG conformance.

Generated from project source, README, and screenshots in `docs/screenshots/`.